1. Tipe te vecanta te queues jane:

- Randomized queues
- Secondary queues
- Collection queues
- Priority queues

2. Metodat te cilat perdoren ne API te Symbol Tables (ST) jane:

- put(key)
- get(key, value)
- size()
- contains(key)

3. Per cdo x dhe y, Java requirement per t'u plotesuar vetite simetrike dhe refleksive jane:

- x.equals(y) = true dhe x.equals(y) nese y.equals(x)
- y.equals(null) = false dhe y.equals(y) nese y.equals(x)
- x.equals(x) = false dhe x.equals(null) nese null.equals(x)
- x.equals(x) = true dhe x.equals(y) nese y.equals(x)

4. Nje BST Node perbehet nga disa elemente:

- Nje key dhe value
- Vetem key
- Referencen per nyjen e majte, te djathte dhe prindin
- Referencen per nyjen e mesit

5. Balancim perfekt nenkuptojme:

- Kur trungu eshte komplet dhe ka performance lineare
- Kur cdo rruge nga root deri tek null link ka kompleksitet kohor konstant
- Kur numri i nyjeve ne subtrungun e majte eshte me i vogel se ne te djathtin
- Kur cdo rruge prej root deri tek null link ka gjatesi te njejte

6. Strategjite per ballafaqim me kolizione qe perdoren ne open addressing jane:

- Cuckoo hashing
- Double hashing
- Linear probing
- Primary clustering

7. Metodat qe e bejne krahasimin e objekteve jane:

- exchange()
- equals()
- compareTo()
- less()

8. Cilat nga sortimet e meposhtme jane algoritme in-place:

- Merge Sort
- Shell Sort
- Insertion Sort
- Selection Sort

9. Operacionet themelore te nje stack dhe queue jane:

- Iterate
- Remove
- Insert
- Size()

10. Tek Maximum Binary Heap:

- Celesi me vleren me te madhe eshte ne poziten q[1]
- Root i pemes eshte gjithmone elementi ne poziten q[1]
- Ne thirrjen e metodes delMax elementi i pare qe largohet eshte ai ne poziten q[n]
- Metoda sink() thirret kur celesi ka vlere me te vogel se njeri nga femijet e tij

11. Insertion Sort karakterizohet me keto veti:

- Ne cdo iterim fiksohet nje element ne poziten e tij perkatese
- Ka numer te madh te shkembimeve dhe numer te vogel te krahasimeve
- Ka numer te vogel te shkembimeve dhe numer te madh te krahasimeve
- Pointeri leviz gjithmone majtas

12. Binary Search Tree kur implementohet ne Java ka celesa:

- Simetrik ne rradhitje
- Generik
- Comparable
- Subtree node based

13. Tek listat, kur fshihet nje element:

- Duhet fshire referenca prej elementit qe do te fshihet
- Duhet fshire edhe referenca edhe te dhenat nga elementi ne liste
- Duhet fshire referenca qe dergon tek ai element nga elementi paraprak
- Garbage collector i Java-s perkujdeset per elementin e fshire

14. Kur nje pjese e array eshte vecse e sortuar, preferohet te perdoret:

- Quicksort
- Shellsort
- Insertion sort
- Selection sort

15. Kerkimi sekvencial ne nje Symbol Table implementohet me:

- Comparable array
- Linked List
- compareTo()
- equals()

16. Pjeset perberese te nje objekti ne Doubly Linked List jane:

- Reference item
- Node item
- Node previous
- Node next

17. Cilat key operations perdoren tek Symbol Table:

- enqueue, dequeue
- add, contains, delete
- put, get, delete
- push, pop

18. Metodat qe perdoren ne API te Priority Queue jane:

- insert(Key key, Value val)
- insert(Key v)
- isEmpty()
- delMax()

19. Priority Queue eshte:

- data type
- collection of objects
- data structure
- sorting algorithm

PJESA II – PYETJE SAKT / PASAKT (S / P)

1. **Te gjitha elementet gjenerike duhet te implementojne Comparable**

PASAKT – Jo domosdoshmërisht, nëse nuk përdoret krahasim i renditjes, nuk është e nevojshme.

2. **Në Priority Queue, elementi i pare ka gjithmone prioritet.**

PASAKT – Varet nga lloji i prioritetit (minPQ, maxPQ).

3. **Metoda contains(key) eshte e tipit Boolean dhe teston nese nje vlere korrespondon me celesin e dhene**

PASAKT – contains(key) (ose containsKey(key)) teston nëse ekziston ky çelës në tabelë, jo nëse ndonjë vlerë i korrespondon atij çelësi. Vlera mund të gjendet me get(key) dhe krahasohet veçmas.

4. **Metoda search(key) tek BST ka numer krahasimesh te barabarte me 1 + thellesine e nyjes**

SAKT – Në kërkimin në BST, numri i krahasimeve në një kërkim të suksesshëm është afërsisht 1 + thellësia e nyjes (sepse krahasohet në çdo nivel, duke filluar nga rrënja).

5. **Binary Search (ordered array) eshte me i shpejte sesa Binary Search Tree**

PASAKT – Binary Search në ordered array ka kompleksitet O(log N). Binary Search Tree (i balancuar) gjithashtu ka O(log N).

6. **Operacioni Insert ne nje 2-node ndan linkun ne mes elementit ne node dhe krijon link me paraardhesin**

PASAKT – Ky përshkrim i përket B-Tree ose 2-3 Tree (ose pjesërisht RBT), ku një 2-node (me një çelës) nga insertimi mund të shndërrohet në 3-node ose të ndahet nëse është overflow.

7. **Hash tables dallojne nga BST dhe Red-Black Trees per nga menyra e qasjes ne elemente**

SAKT – Hash tables qasen në elemente duke përdorur **funksion hash** → qasje në kohë O(1) mesatar. BST dhe Red-Black Trees përdorin **krahasim çelësash** dhe strukturë pemore → O(log n) për qasje.

8. **Primary Clustering vjen si pasoje e Linear Probing tek closed addressing**

PASAKT – Closed addressing (separate chaining) nuk përdor linear probing dhe nuk krijon primary clustering. Primary clustering është problem i open addressing, konkretisht i linear probing, ku elementet grumbullohen në vargje të njëpasnjëshme.

9. **Order of growth eshte me i miri tek funksionet eksponenciale**

PASAKT – Funksionet eksponenciale (p.sh. 2^n) rriten shumë shpejt, janë ndër më të këqijtë për sa i përket kompleksitetit kohor. Më të mirët janë: konstant O(1), logaritmik O(log n), linear O(n), etj.

10. **Stacks lejojne qasje vetem ne nje element: elementin e fundit te insertuar**

SAKT – Një stack është strukturë e të dhënave LIFO (Last In, First Out).

11. **Lartesia e 2-3 Tree ne rastin me te keq eshte e njejte me lartesine ne rastin me te mire**

SAKT – 2–3 Trees janë gjithmonë të balancuara, të gjitha rrugët nga root deri te leaf kanë gjatësi të njëjtë.

12. Insert tek Priority Queue merr me pak kohe sesa delMax tek ordered array.

SAKT – Në Priority Queue (me heap), insert është O(log n), ndërsa delMax në ordered array është O(n) .

13. Hash tables mund te perdoren per te marre te dhena te renditura

PASAKT – Hash tables nuk ruajnë rendin e çelësave dhe nuk janë të dizajnuara për të dhëna të renditura. Për të marrë të dhëna sipas radhës, përdoren struktura si BST, Red-Black Tree, ose hash table me indeksim shtesë.

14. Open addressing njihet ndryshe edhe si closed hashing

SAKT – Open addressing = closed hashing

15. Symbol Table paraqet algoritme per kerkim

SAKT – Symbol Table është një **abstraksion** që përmban operacione (get, put, delete) dhe mund të implementohet me algoritme të ndryshme kërkimi (BST, hashing, lista). Pra, vetë është strukturë, jo algoritëm, por përmban algoritme kërkimi në implementim.

16. Insertimi dhe fshirja tek BST eshte me e shpejte sesa tek Red-Black Tree

PASAKT – BST i pabalancuar mund të ketë rastin më të keq O(n), ndërsa Red-Black Tree është i balancuar dhe garanton O(logn).

17. Hash tables jane struktura te te dhenave qe implementojne Symbol Tables

SAKT – Po, një nga implementimet më të shpeshta të Symbol Table është me hash table.

18. Per te kerkuar ne varg me BST inputi duhet te jete i radhitur

PASAKT – Për të kërkuar në një varg duke përdorur Binary Search Tree, nuk është e nevojshme që inputi të jetë i radhitur fillimisht.

19. Priority Queue aplikohet ne statistics, operating systems dhe computer networks

SAKT – Statistics (p.sh. gjetja e k elementëve më të mëdhenj), Operating Systems (scheduling bazuar në prioritet), Computer Networks (trajtimi i paketave me prioritet)

20. Metoda get() sherben per kerkim tek Binary Heap

PASAKT – Në Binary Heap nuk ka metodë të quajtur get() për kërkim të rastësishëm. Qasja është përmes peek() ose search.

21. Per te llogaritur hash funksionin perdoret metoda e Hornerit

SAKT – Metoda e Hornerit përdoret për llogaritjen e vlerës së polinomit dhe kur bëjmë hash për stringje.

22. Tek hash tables elementet ruhen ne nje key-indexed table

PASAKT – Hash tables jo gjithmonë janë *key-indexed table* në kuptimin e thjeshtë. Një *key-indexed table* është një tabelë ku çelësat janë indeks direkt (p.sh. indeks numerik i vogël). Hash tables përdorin një funksion hash për të kthyer çelësat arbitrar në indekse dhe kanë mekanizma për trajtimin e kolizioneve.

23. Priority Queue paraqet algoritëm për sorting (searching algorithm)

PASAKT – Priority Queue është strukturë e të dhënave, jo algoritëm. Ajo mund të përdoret për sorting (p.sh. Heap Sort), por vetë nuk është algoritëm për sorting apo searching.